Rebuild Guide for Bobby Brock Insurance

Critical Implementation Priorities

1. JSX Runtime Configuration

To prevent JSXDEV error loops, implement the following:

```typescript
// vite.config.ts
export default defineConfig({
  plugins: [
    react({
      // Explicitly set JSX runtime
      jsxRuntime: 'automatic',
      // Ensure proper React imports
      babel: {
        plugins: [
          ['@babel/plugin-transform-react-jsx', {
            runtime: 'automatic'
          }]
        ]
      }
    })
  ]
})
```

Key Prevention Steps:

1. Use explicit JSX runtime configuration
2. Avoid mixing JSX runtime modes
3. Ensure consistent React imports
4. Use proper module resolution
5. Implement proper error boundaries
6. Use TypeScript for type safety

2. SEO Structure Implementation

Page Hierarchy

```unset
/
├── understand-medicare/                    # Primary Medicare content
|   ├── basics/                  # Foundational knowledge
|   ├── eligibility/            # Eligibility requirements
```

```
|   ├── parts/                  # Medicare parts
|   └── supplement/             # Supplement plans
|
├── medicare-plans/                     # Insurance plans
|   ├── advantage/              # Medicare Advantage
|   ├── supplement/             # Medigap plans
|   └── prescription/           # Part D plans
|
├── resources/                  # Educational resources
|   ├── guides/                 # Detailed guides
|   ├── calculators/            # Cost calculators
|   └── glossary/              # Medicare terminology
|
├── blog/                       # Educational blog
└── videos/                     # Video content
```

SEO Implementation

1. **Meta Structure**

```typescript
// Example meta configuration
const metaConfig = {
  title: {
    template: '%s | Bobby Brock Insurance',
    default: 'Medicare Insurance Plans & Coverage | Bobby Brock Insurance'
  },
  description: {
    default: 'Expert Medicare insurance guidance and plan selection in Tupelo, MS. Compare
Medicare Advantage, Supplement, and Part D plans.'
  },
  openGraph: {
    type: 'website',
    locale: 'en_US',
    site_name: 'Bobby Brock Insurance'
  }
}
```

2. **Content Structure**

    - Each page should have:
        - Clear H1 hierarchy
        - Structured content sections
        - Internal linking
        - Schema markup
        - Breadcrumb navigation

3. **Technical SEO**

    - Implement proper canonical URLs
    - Use semantic HTML5
    - Ensure mobile responsiveness
    - Optimize page load speed
    - Implement proper sitemap

4. **Performance Optimization**

    - Use React.lazy for code splitting
    - Implement proper loading states
    - Optimize images and media
    - Use proper caching strategies
    - Minimize bundle size

Overview

This guide provides step-by-step instructions for rebuilding the Bobby Brock Insurance website using the preserved code snippets and documentation. The rebuild should prioritize stability, performance, and maintainability while preserving existing functionality.

Core Dependencies

```JSON
{
  "dependencies": {
    "contentful": "^10.0.0",
    "react": "^18.0.0",
    "react-dom": "^18.0.0",
    "typescript": "^5.0.0"
  }
}
```

Implementation Steps

1. Contentful Setup

1. Initialize Contentful client using the `ContentfulService` class from `contentful-fetch.ts`
2. Configure environment variables:

```Unset
CONTENTFUL_SPACE_ID=your_space_id
CONTENTFUL_ACCESS_TOKEN=your_access_token
CONTENTFUL_ENVIRONMENT=master
```

3. Verify content types match those in `contentful-fields.md`
4. Test fetch methods with sample content

2. Schema Implementation

1. Implement `SchemaGenerator` class from `schema-generators.ts`
2. Create schema injection utility using `inject-schema-core.ts`
3. Add schema validation for:
    - Video pages
    - Blog posts
    - Foundational pages
    - FAQ sections
4. Test schema generation with sample data

3. SEO Implementation

1. Implement `SEO` component from `seo-snippets.tsx`
2. Configure meta tags for:
    - Basic pages
    - Blog posts
    - Video pages
    - Resource pages
3. Set up canonical URLs
4. Implement Open Graph and Twitter Card meta tags

4. Routing Structure

1. Implement routes as documented in `routing-structure.md`:

```
Unset
/videos/[slug]
/blog/[slug]
/resources
/medicarepartd
/medicaresupplement
```

2. Set up dynamic route handlers
3. Implement breadcrumb navigation
4. Configure 404 and error pages

5. Content Templates

1. Implement video template from `example-template-video.tsx`
2. Create corresponding templates for:
    - Blog posts
    - Resource pages
    - Foundational pages

3. Ensure responsive design
4. Implement accessibility features

6. Team Structure Integration

1. Implement team data structure from `site-structure.md`
2. Create team member components
3. Set up team page routing
4. Implement team member filtering/sorting

Directory Structure Implementation

Overview

The site follows a hierarchical structure that maps to Contentful content types and supports SEO-friendly URLs. Each directory corresponds to specific content types and their respective slugs.

Core Directory Tree

```
Unset
/
├── medicare/                       # Medicare education content
│   ├── basics/                     # foundationalPage.pageSlug = "basics"
│   ├── eligibility/                # foundationalPage.pageSlug = "eligibility"
│   ├── parts/                      # Medicare parts overview
│   │   ├── part-a/                 # foundationalPage.pageSlug = "part-a"
│   │   ├── part-b/                 # foundationalPage.pageSlug = "part-b"
│   │   ├── part-c/                 # foundationalPage.pageSlug = "part-c"
│   │   ├── part-d/                 # foundationalPage.pageSlug = "part-d"
│   │   ├── supplement/             # foundationalPage.pageSlug = "supplement"
│   │   ├── compare/                # Plan comparison tools
│   │   ├── faqs/                   # Frequently asked questions
│   |
│   ├── plans/                      # Insurance plan information
│   │   ├── advantage/              # Medicare Advantage plans
│   │   │   ├── hmo/                # foundationalPage.pageSlug = "hmo"
│   │   │   ├── ppo/                # foundationalPage.pageSlug = "ppo"
│   │   │   ├── supplement/         # Medigap plans
│   │   │   ├── prescription/       # Part D plans
│   │   │   ├── dental/             # Dental coverage
│   │   │   └── vision/             # Vision coverage
│   |
│   ├── resources/                  # Educational resources
│   │   ├── guides/                 # Educational guides
│   │   │   └── [slug]              # foundationalPage.pageSlug (dynamic)
│   │   ├── calculators/            # Cost calculators
│   │   ├── glossary/               # Medicare terminology
│   │   └── state-specific/         # State-specific information
│   |
```

```
|   ├── blog/                        # Blog content
|   |   ├── [slug]                   # pageBlogPost.slug
|   |
|   ├── videos/                      # Video content
|   |   ├── watch/                   # Video player pages
|   |   |   └── [slug]               # video.slug
|   |
|   ├── team/                        # foundationalPage.pageSlug = "team"
|   ├── contact/                     # foundationalPage.pageSlug = "contact"
|   ├── about/                       # foundationalPage.pageSlug = "about"
|   ├── appointment/                 # External URL to scheduling
|   └── privacy-policy/              # foundationalPage.pageSlug = "privacy-policy"
```

Implementation Notes

Contentful Integration

1. Map each directory to its corresponding Contentful content type:

    - `foundationalPage` for static pages
    - `pageBlogPost` for blog entries
    - `video` for video content

2. Implement dynamic routing for:

    - Blog posts: `/blog/[slug]`
    - Videos: `/videos/watch/[slug]`
    - Resource guides: `/resources/guides/[slug]`

SEO Considerations

1. Ensure each page has:

    - Proper meta tags
    - Structured data
    - Breadcrumb navigation
    - Canonical URLs

2. Implement dynamic sitemap generation based on:

    - Contentful entries
    - Directory structure
    - Last modified dates

Performance Optimization

1. Implement route-based code splitting
2. Use static generation for foundational pages

3. Implement ISR (Incremental Static Regeneration) for:
    - Blog posts
    - Resource guides
    - Video pages

Content Management

1. Maintain Contentful content types:

    - `foundationalPage`: Static pages
    - `pageBlogPost`: Blog content
    - `video`: Video content
    - `resourceGuide`: Educational guides

2. Implement content validation:

    - Required fields
    - Slug uniqueness
    - URL structure compliance

Navigation Implementation

1. Create hierarchical navigation components:

    - Main navigation
    - Footer navigation
    - Breadcrumbs
    - Sidebar navigation (where applicable)

2. Implement dynamic navigation based on:

    - Directory structure
    - Contentful entries
    - User permissions

Critical Implementation Notes

Contentful Fetch Safety

```typescript
// Always implement error boundaries
try {
  const content = await contentfulService.getContent();
} catch (error) {
  // Log error
  // Return fallback content
  // Notify monitoring
}
```

Schema Validation

```typescript
// Validate schemas before injection
if (SchemaInjector.validateSchema(schema)) {
  SchemaInjector.getInstance().injectSchema({ schema });
}
```

SEO Best Practices

- Implement dynamic meta tags
- Use semantic HTML
- Include structured data
- Optimize for social sharing

Performance Considerations

- Implement lazy loading for images/videos
- Use proper caching strategies
- Optimize bundle size
- Implement proper code splitting

Testing Requirements

Contentful Tests

1. Test all fetch methods
2. Verify error handling
3. Test content type validation
4. Verify field mapping

Schema Tests

1. Validate schema generation
2. Test schema injection
3. Verify JSON-LD formatting
4. Test error cases

SEO Tests

1. Verify meta tag generation
2. Test canonical URLs
3. Validate structured data
4. Check social media tags

Deployment Checklist

1. Verify environment variables
2. Test contentful connections

3. Validate schema generation
4. Check SEO implementation
5. Verify routing
6. Test performance metrics

Maintenance Guidelines

1. Keep content types in sync with Contentful
2. Update schemas as needed
3. Maintain SEO best practices
4. Monitor performance metrics

Error Handling

Implement comprehensive error handling for:

- Contentful fetch failures
- Schema generation errors
- Routing issues
- SEO implementation problems

Monitoring Setup

1. Set up error tracking
2. Implement performance monitoring
3. Configure SEO monitoring
4. Set up content delivery monitoring

Security Considerations

1. Validate all inputs
2. Implement proper CORS
3. Secure API endpoints
4. Protect sensitive data

Performance Optimization

1. Implement proper caching
2. Optimize images/videos
3. Minimize bundle size
4. Use proper code splitting

Accessibility Requirements

1. Follow WCAG 2.1 guidelines
2. Implement proper ARIA labels
3. Ensure keyboard navigation
4. Test with screen readers

Documentation Updates

1. Keep API documentation current
2. Update content model docs
3. Maintain deployment guides
4. Document new features

Support Contact

For implementation questions or issues:

- Technical Support: [contact info]
- Contentful Support: [contact info]
- SEO Support: [contact info]

Schema Implementation

Overview

This section details the implementation of structured data (JSON-LD) following Google's standards for Medicare and insurance content. Each content type requires specific schema markup for optimal SEO and rich results.

Content Type Schemas

Foundational Pages

```JSON
{
  "@context": "https://schema.org",
  "@type": "WebPage",
  "name": "Page Title",
  "description": "Page description",
  "url": "https://tupelomedicareconnect.com/[path]",
  "mainEntity": {
    "@type": "Article",
    "headline": "Page Title",
    "author": {
      "@type": "Organization",
      "name": "Bobby Brock Insurance",
      "url": "https://tupelomedicareconnect.com"
    },
    "publisher": {
      "@type": "Organization",
      "name": "Bobby Brock Insurance",
      "logo": {
        "@type": "ImageObject",
        "url": "https://tupelomedicareconnect.com/logo.png"
      }
```

```
    },
    "datePublished": "YYYY-MM-DD",
    "dateModified": "YYYY-MM-DD"
  }
}
```

Blog Posts

```JSON
{
  "@context": "https://schema.org",
  "@type": "BlogPosting",
  "headline": "Blog Post Title",
  "description": "Blog post description",
  "image": "Featured image URL",
  "author": {
    "@type": "Person",
    "name": "Author Name"
  },
  "publisher": {
    "@type": "Organization",
    "name": "Bobby Brock Insurance",
    "logo": {
      "@type": "ImageObject",
      "url": "https://tupelomedicareconnect.com/logo.png"
    }
  },
  "datePublished": "YYYY-MM-DD",
  "dateModified": "YYYY-MM-DD",
  "mainEntityOfPage": {
    "@type": "WebPage",
    "@id": "https://tupelomedicareconnect.com/blog/[slug]"
  }
}
```

Videos

```JSON
{
  "@context": "https://schema.org",
  "@type": "VideoObject",
  "name": "Video Title",
  "description": "Video description",
  "thumbnailUrl": "Thumbnail image URL",
  "uploadDate": "YYYY-MM-DD",
  "duration": "PT5M30S",
```

```
  "contentUrl": "Video URL",
  "embedUrl": "Embed URL",
  "publisher": {
    "@type": "Organization",
    "name": "Bobby Brock Insurance",
    "logo": {
      "@type": "ImageObject",
      "url": "https://tupelomedicareconnect.com/logo.png"
    }
  }
}
```

Medicare Plans

```JSON
{
  "@context": "https://schema.org",
  "@type": "InsuranceAgency",
  "name": "Bobby Brock Insurance",
  "description": "Medicare insurance plans and coverage",
  "url": "https://tupelomedicareconnect.com",
  "logo": "https://tupelomedicareconnect.com/logo.png",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "123 Main St",
    "addressLocality": "Tupelo",
    "addressRegion": "MS",
    "postalCode": "38801",
    "addressCountry": "US"
  },
  "offers": {
    "@type": "Offer",
    "itemOffered": {
      "@type": "InsurancePlan",
      "name": "Plan Name",
      "description": "Plan description",
      "provider": {
        "@type": "Organization",
        "name": "Insurance Provider"
      }
    }
  }
}
```

Implementation Guidelines

Schema Injection

1. Use the `SchemaGenerator` class for dynamic schema generation
2. Inject schemas using the `SchemaInjector` utility
3. Validate schemas before injection
4. Implement error handling for schema generation

Content Type Mapping

1. Map each content type to its corresponding schema:
    - `foundationalPage` → WebPage + Article
    - `pageBlogPost` → BlogPosting
    - `video` → VideoObject
    - Plan pages → InsuranceAgency + InsurancePlan

Required Fields

1. All schemas must include:

    - `@context`
    - `@type`
    - `name`
    - `description`
    - `url`
    - `publisher` information

2. Time-sensitive content must include:

    - `datePublished`
    - `dateModified`

Validation

1. Use Google's Rich Results Test tool
2. Validate schemas before deployment
3. Monitor schema errors in Google Search Console
4. Update schemas as needed based on Google's guidelines

Best Practices

SEO Optimization

1. Use descriptive, keyword-rich names and descriptions
2. Include relevant images and media
3. Maintain consistent URL structures
4. Update modification dates when content changes

Performance

1. Minimize schema size
2. Use appropriate nesting levels
3. Avoid redundant information
4. Cache generated schemas when possible

Maintenance

1. Regular schema validation
2. Update schemas with content changes
3. Monitor Google Search Console for errors
4. Keep up with schema.org updates

Error Handling

```typescript
try {
  const schema = SchemaGenerator.generateSchema(content);
  if (SchemaValidator.validate(schema)) {
    SchemaInjector.inject(schema);
  } else {
    // Log validation errors
    // Fall back to basic schema
  }
} catch (error) {
  // Log error
  // Inject minimal schema
}
```

Testing

1. Test each content type's schema
2. Verify rich results in Google Search Console
3. Check schema validation
4. Test error handling scenarios